hw7

Problem 1

```
n = 6;
lambda_exact = @(j) 4 * (sin(j * pi / 14).^2);
col_vec = (1:n)';
evec_exact = @(j) sqrt(2 / 7) * sin((col_vec * j * pi) / 7);

T = gallery('tridiag', 6, -1, 2, -1);

for j = 1:n
    v = rand(6, 1);
    v_0 = v / norm(v);
    meu = lambda_exact(j);
    w = (T - meu * eye(6)) \ v_0;

    fprintf("Eigenvalue/vector %d:\n", j);
    calc_evec = w / norm(w)
    exact_evec = evec_exact(j)

    if sign(calc_evec(1)) ~= sign(exact_evec(1))
        calc_evec = -calc_evec;
    end
    diff_vec = exact_evec - calc_evec

    calc_eval = calc_evec.' * T * calc_evec
end
```

Output

```
[Warning: Matrix is close to singular or badly scaled. Results may be
inaccurate. RCOND =  2.891858e-18.]
[> In hw1 (line 13)
]
Eigenvalue/vector 1:

calc_evec =

    0.2319
    0.4179
    0.5211
    0.5211
```

```
    0.4179
    0.2319


exact_evec =

    0.2319
    0.4179
    0.5211
    0.5211
    0.4179
    0.2319


diff_vec =

   1.0e-16 *

         0
         0
         0
         0
    0.5551
    0.5551


calc_eval =

    0.1981

[Warning: Matrix is close to singular or badly scaled. Results may be
inaccurate. RCOND =  3.660947e-17.]
[> In hw1 (line 13)
]
Eigenvalue/vector 2:

calc_evec =

   -0.4179
   -0.5211
   -0.2319
    0.2319
    0.5211
    0.4179
```

```
exact_evec =

    0.4179
    0.5211
    0.2319
   -0.2319
   -0.5211
   -0.4179


diff_vec =

   1.0e-14 *

   -0.1166
   -0.1998
   -0.2193
   -0.2331
   -0.2220
   -0.0722


calc_eval =

    0.7530

[Warning: Matrix is close to singular or badly scaled. Results may be
inaccurate. RCOND =  1.060924e-16.]
[> In hw1 (line 13)
]
Eigenvalue/vector 3:

calc_evec =

    0.5211
    0.2319
   -0.4179
   -0.4179
    0.2319
    0.5211


exact_evec =

    0.5211
    0.2319
```

```
   -0.4179
   -0.4179
    0.2319
    0.5211


diff_vec =

   1.0e-15 *

    0.2220
    0.3886
    0.1665
    0.1110
   -0.3608
    0.1110


calc_eval =

    1.5550

[Warning: Matrix is close to singular or badly scaled. Results may be
inaccurate. RCOND =  5.052017e-17.]
[> In hw1 (line 13)
]
Eigenvalue/vector 4:

calc_evec =

   -0.5211
    0.2319
    0.4179
   -0.4179
   -0.2319
    0.5211


exact_evec =

    0.5211
   -0.2319
   -0.4179
    0.4179
    0.2319
   -0.5211
```

```
diff_vec =

   1.0e-15 *

   -0.3331
   -0.2776
         0
   -0.0555
    0.0278
   -0.3331


calc_eval =

    2.4450

[Warning: Matrix is close to singular or badly scaled. Results may be
inaccurate. RCOND =  3.660947e-17.]
[> In hw1 (line 13)
]
Eigenvalue/vector 5:

calc_evec =

    0.4179
   -0.5211
    0.2319
    0.2319
   -0.5211
    0.4179


exact_evec =

    0.4179
   -0.5211
    0.2319
    0.2319
   -0.5211
    0.4179


diff_vec =
```

```
   1.0e-15 *

    0.3331
    0.5551
   -0.6661
    0.7216
   -0.2220
   -0.4441


calc_eval =

    3.2470

[Warning: Matrix is close to singular or badly scaled. Results may be
inaccurate. RCOND =  5.205345e-17.]
[> In hw1 (line 13)
]
Eigenvalue/vector 6:

calc_evec =

   -0.2319
    0.4179
   -0.5211
    0.5211
   -0.4179
    0.2319


exact_evec =

    0.2319
   -0.4179
    0.5211
   -0.5211
    0.4179
   -0.2319


diff_vec =

   1.0e-14 *

   -0.1138
    0.0777
```

```
    -0.0444
    -0.0333
     0.0222
    -0.1193


calc_eval =

    3.8019
```

Despite the matrix being singular and the Matlab warning, the calculated eigenvector was really accurate

Problem 2

Problem 2. Let $A \in R^{m \times m}$ be upper Hessenberg, that is, $a_{ij} = 0$ for $i > j + 1$. Assume that a QR step without shift is applied to A, that is,

$$A = QR, \quad B = RQ,$$

where QR in the first equation is a QR factorization of A. Describe the computation of R and B using Givens rotations. Show that B is upper Hessenberg and that cost of computing B is proportional to m^2.

Let $\ldots_1$ be the givens applied to A to get R so $\ldots_1 = R$.
What we can do instead is apply the same rotations transposed to R but multiply on the right side instead. So something like $R_{T1}\ldots_T$

We have m-1 Givens matricies (because upper hessenberg, m-1 elements to be eliminated). Each rotation operates on each row. Each Givens matrix affects at most m elements (since each matrix affects a row and there are m rows) so we get m(m-1) ~ m^2
We can use this same logic for computing B. Each rotation operates on m elements of a col and this is done m-1 times so ~m^2

Know that affects position (i+1,i) and everything above it so $_T$ will also also affect position (i+1,i) and everything to the right. Since R is upper triangular, we know that R_{T1} will be upper triangular except for (2,1) which is upper hessenberg. Same with R_{T1T2}. So $R_{T1}\ldots_T$ will be upper hessenberg
The BR=RQR=RA. So we have BR=RA so we have B=RAR^-1
Since we have R upper triangular and A upper Hessenberg, RA is upper Hessenberg, which means that RAR^-1 is upper Hessenberg for the same reasons so B is upper Hessenberg

Problem 3

Problem 3. (We showed in class that if Givens rotations are used at each step of the QR algorithm without shifts, then the symmetric tridiagonal form of a matrix is preserved. The purpose of this problem is to show that the symmetric tridiagonal form is preserved for an unreduced symmetric tridiagonal matrix regardless of how QR factorization is computed at each step of the QR algorithm.)

Definition. A symmetric tridiagonal $T \in R^{m\times m}$ is unreduced if all its elements on the first subdiagonal (equivalently all elements on the first superdiagonal) are nonzero.

Assume that $T \in R^{m\times m}$ is symmetric tridiagonal and unreduced and let $T = QR$ be a QR factorization of T, that is, $Q \in R^{m\times m}$ is orthogonal and $R \in R^{m\times m}$ is real upper triangular.

(a) Show that the first $m-1$ columns of R are linearly independent. **Hint:** First show that the first $m-1$ columns of T are linearly independent.

(b) Let $\hat{T}, \hat{Q} \in R^{m\times(m-1)}$ consist of the first $m-1$ columns of T and Q, respectively, and let $\hat{R} \in R^{(m-1)\times(m-1)}$ be obtained from R by deleting its last column and last row. Show that $\hat{R}$ is nonsingular and that $\hat{T} = \hat{Q}\hat{R}$. Using these facts prove that Q is upper Hessenberg.

(c) Show that RQ is tridiagonal. **Hint:** First show that RQ is upper Hessenberg and then use symmetry of RQ.

a)

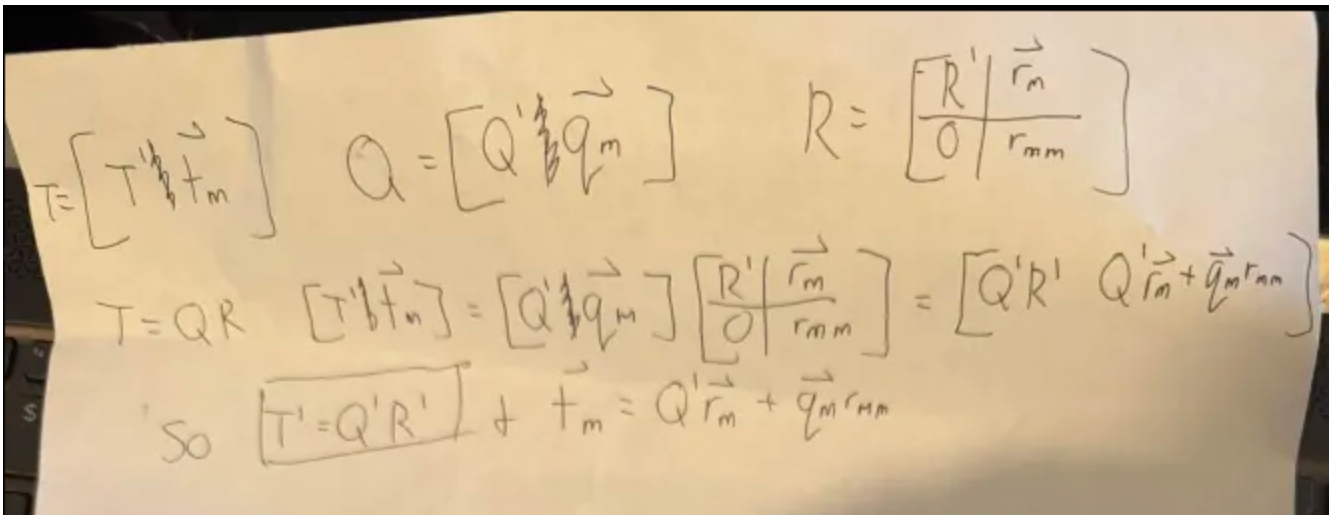
Let T' and R' denote the first m-1 columns of T and R respectively. Then we know if T' is linearly independent, then R' is linearly independent (To show this proof by contradiction, let T' be linearly independent where R' is not. Then let x be nonzero such that R'x=0, then T'x=QR'x=Q0=0 but T' is linearly independent so T'x cannot equal 0 so contradiction. So if T' is linearly indep, then R' must be as well)

Showing T' is linearly indep:
If we remove the first row of T' (call this T''), we get an upper-triangular matrix where the main diagonal is the subdiagonal of T. Since all subdiagonal elements are nonzero, we know the main diagonal of T'' is all nonzero so det(T'') =/= 0. So the cols of T'' are linearly independent. Then if we add the row back in, we know that increasing the size of linearly independent vectors retains linear independence so the cols of T' must be linearly independent as well

Therefore we know T' is linearly indep, so R' is linearly independent

b)

For this problem, call $\hat{T}, \hat{Q}, \hat{R}$ T', Q', and R' respectively.



We have already shown that T' is linearly independent. From here we can show R is nonsingular by contradiction.
Let R' be singular, then there exists an x such that R'x=0 where x is not the 0 vector
Since T' is nonsingular, $T'x \neq 0$ since x is not the 0 vector.
But Q'R'x=Q0=0 which is a contradiction. So $R'x \neq 0$ for all $x \neq 0$ so R' is linearly independent and therefore nonsingular

$Q' = T'R'^{-1}$, know that R'^{-1} is upper triangular and that T' is symmetric triadiagonal. A tridiagonal matrix * an upper triangular matrix is upper Hessenberg so we are done

c)

Since we know Q' is upper Hessenberg, adding a column retains this property so Q is upper Hessenberg. R is upper triangular so then RQ must be upper Hessenberg.

Now we need to show $RQ = Q^T R^T$
Because T is symmetric tridiagonal
$T = T^T \implies QR = R^T Q^T \implies R = Q^T R^T Q^T \implies RQ = Q^T R^T$
So we have $RQ = (RQ)^T$ so RQ is symmetric.

Since RQ is symmetric and upper Hessenberg, it must be symmetric tridiagonal

Problem 4

Code:

```
T = gallery('tridiag', 6, -1, 2, -1);

lambda_exact = @(j) 4 * (sin(j * pi / 14).^2);
evals_exact = [lambda_exact(1); lambda_exact(2); lambda_exact(3);
lambda_exact(4); lambda_exact(5); lambda_exact(6)];
expected_A = diag(evals_exact);
```

```
disp(full(expected_A))
A = T;
for k=1:50
    [Q, R] = qr(A);
    A = R * Q;
    if mod(k, 10) == 0
        fprintf("Iteration %d:\n", k);
        disp(full(A));
    end
end

A = T;
for k=1:3
    a_5 = A(5,5);
    b_5 = A(5,6);
    a_6 = A(6,6);

    delta = (a_5 - a_6) / 2;

    if delta == 0
        sgn = 1;
    else
        sgn = sign(delta);
    end
    meu = a_6 - (sgn * b_5^2) / (abs(delta) + sqrt(delta^2 + b_5^2));

    [Q, R] = qr(A - meu * eye(6));
    A = R * Q + meu * eye((6));

    fprintf("Iteration %d:\n", k);
    disp(full(A));
end
```

Output:

```
    0.1981         0         0         0         0         0
         0    0.7530         0         0         0         0
         0         0    1.5550         0         0         0
         0         0         0    2.4450         0         0
         0         0         0         0    3.2470         0
         0         0         0         0         0    3.8019

Iteration 10:
    3.7337   -0.1842    0.0000   -0.0000   -0.0000    0.0000
   -0.1842    3.2884   -0.1481   -0.0000    0.0000   -0.0000
         0   -0.1481    2.4706   -0.0342   -0.0000   -0.0000
```

```
         0         0   -0.0342    1.5562   -0.0017    0.0000
         0         0         0   -0.0017    0.7530   -0.0000
         0         0         0         0   -0.0000    0.1981

Iteration 20:
    3.7987   -0.0424    0.0000    0.0000   -0.0000    0.0000
   -0.0424    3.2501   -0.0084   -0.0000    0.0000   -0.0000
         0   -0.0084    2.4451   -0.0004   -0.0000   -0.0000
         0         0   -0.0004    1.5550   -0.0000    0.0000
         0         0         0   -0.0000    0.7530   -0.0000
         0         0         0         0   -0.0000    0.1981

Iteration 30:
    3.8018   -0.0088    0.0000    0.0000   -0.0000    0.0000
   -0.0088    3.2471   -0.0005   -0.0000    0.0000   -0.0000
         0   -0.0005    2.4450   -0.0000   -0.0000   -0.0000
         0         0   -0.0000    1.5550   -0.0000    0.0000
         0         0         0   -0.0000    0.7530   -0.0000
         0         0         0         0   -0.0000    0.1981

Iteration 40:
    3.8019   -0.0018    0.0000    0.0000   -0.0000    0.0000
   -0.0018    3.2470   -0.0000   -0.0000    0.0000   -0.0000
         0   -0.0000    2.4450   -0.0000   -0.0000   -0.0000
         0         0   -0.0000    1.5550   -0.0000    0.0000
         0         0         0   -0.0000    0.7530   -0.0000
         0         0         0         0   -0.0000    0.1981

Iteration 50:
    3.8019   -0.0004    0.0000    0.0000   -0.0000    0.0000
   -0.0004    3.2470   -0.0000   -0.0000    0.0000   -0.0000
         0   -0.0000    2.4450   -0.0000   -0.0000   -0.0000
         0         0   -0.0000    1.5550   -0.0000    0.0000
         0         0         0   -0.0000    0.7530   -0.0000
         0         0         0         0   -0.0000    0.1981

Iteration 1:
    3.0000    0.7071    0.0000    0.0000         0         0
    0.7071    2.0000    1.2247    0.0000         0    0.0000
         0    1.2247    1.6667   -0.9428         0         0
         0         0   -0.9428    2.3333    0.8660    0.0000
         0         0         0    0.8660    2.0000   -0.5000
         0         0         0         0   -0.5000    1.0000

Iteration 2:
    3.3178    0.4699    0.0000   -0.0000   -0.0000   -0.0000
```

```
    0.4699    2.7971   -0.8137   -0.0000   -0.0000   -0.0000
         0   -0.8137    1.6548    1.3084    0.0000   -0.0000
         0         0    1.3084    1.8419    0.5504   -0.0000
         0         0         0    0.5504    1.6347   -0.0261
         0         0         0         0   -0.0261    0.7537

Iteration 3:
    3.4675    0.3768    0.0000   -0.0000   -0.0000    0.0000
    0.3768    3.0710   -0.5478    0.0000    0.0000    0.0000
         0   -0.5478    2.5853   -0.8413   -0.0000    0.0000
         0         0   -0.8413    0.7159   -0.5010    0.0000
         0         0         0   -0.5010    1.4073    0.0000
         0         0         0         0    0.0000    0.7530
```

It converged to .7530 really quickly, and it was close to some of the others

Problem 5

```
H1 = [1, 2, 3;
      2, 3, 5;
      0, 1, 1];

H2 = [1, 2, 3;
      1, 0, 1;
      0, -2, 2];

for k=1:50
    [Q1, R1] = qr(H1);
    H1 = R1 * Q1;

    [Q2, R2] = qr(H2);
    H2 = R2 * Q2;
end

fprintf("Calculated H1^(50): \n")
disp(H1)
fprintf("Actual evals for H1^(50): \n")
e_h1 = eig(H1)

fprintf("Calculated H2^(50): \n")
disp(H2)
fprintf("Actual evals for H2^(50): \n")
e_h2 = eig(H2)
```

Output:

```
Calculated H1^(50):
    5.3723    4.4907    2.1511
   -0.0000   -0.3723    0.4538
         0         0    0.0000

Actual evals for H1^(50):

e_h1 =

    5.3723
   -0.3723
    0.0000

Calculated H2^(50):
    1.1833    2.5330   -1.9145
   -1.3145    2.9831    0.7293
         0   -0.0000   -1.1663

Actual evals for H2^(50):

e_h2 =

   2.0832 + 1.5874i
   2.0832 - 1.5874i
  -1.1663 + 0.0000i
```